

CHEF

— FOR —

COMPLETE BEGINNERS

From Servers and Networks
to Cloud Infrastructure and Automation



SERVERS



NETWORKS



CLOUD



AUTOMATION

- | | |
|--|---|
| ☒ Infrastructure as Code | ☒ Cloud Ready |
| ☒ Configuration Management | ☒ Scalable Solutions |
| ☒ Idempotent Automation | ☒ Production Best Practices |
| ☒ Repeatable and Reliable | ☒ Real-World Examples |

**Build.
Automate.
Scale.
Repeat.**



BEGINNER
FRIENDLY



PRACTICAL
EXAMPLES



CLOUD
FOCUSED



PRODUCTION
READY

AUTOMATE TODAY. ENGINEER TOMORROW.

1. CHEF AND CONFIGURATION MANAGEMENT

★ WHAT CONFIGURATION MANAGEMENT SOLVES



- Keeps infrastructure consistent.
- Ensures systems are built the same way every time.
- Prevents human mistakes.
- Detects and corrects unwanted changes.
- Saves time, reduces downtime, improves reliability.

MANUAL ADMINISTRATION VS AUTOMATION

MANUAL ADMINISTRATION

- ✗ Login to servers
- ✗ Run commands one by one
- ✗ Easy to forget steps
- ✗ Inconsistent results
- ✗ Hard to scale
- ✗ Prone to human error



AUTOMATION (CHEF)

- ✓ Define once, apply everywhere
- ✓ Repeatable and consistent
- ✓ No manual SSH
- ✓ Tracks desired state
- ✓ Scales to thousands of servers
- ✓ Reduces errors and saves time

CONFIGURATION DRIFT



DESIRED STATE



ACTUAL STATE

When systems change outside of your automation (manual changes, updates, hotfixes), they drift from the desired state.

Chef detects this drift and brings the system back to the desired state.

DESIRED STATE



The state you define in your code (recipes, cookbooks, attributes). This is the "source of truth".

IDEMPOTENCY



Running the same Chef code multiple times should always produce the same result without causing problems. Safe to run again and again.

WHERE CHEF FITS IN DEVOPS



CODE

Infrastructure as Code



CHEF

Configure & Manage



INFRASTRUCTURE

Consistent, Reliable, Scalable



MONITOR

Observe & Detect Issues



IMPROVE

Feedback Loop Continuous Betterment



PRODUCTION NOTE

Chef makes your infrastructure predictable. Predictable systems are scalable, secure, and easy to operate.

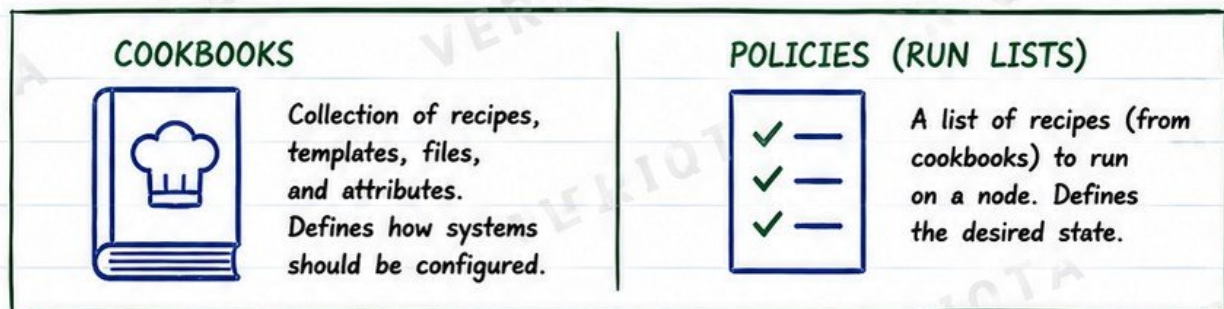
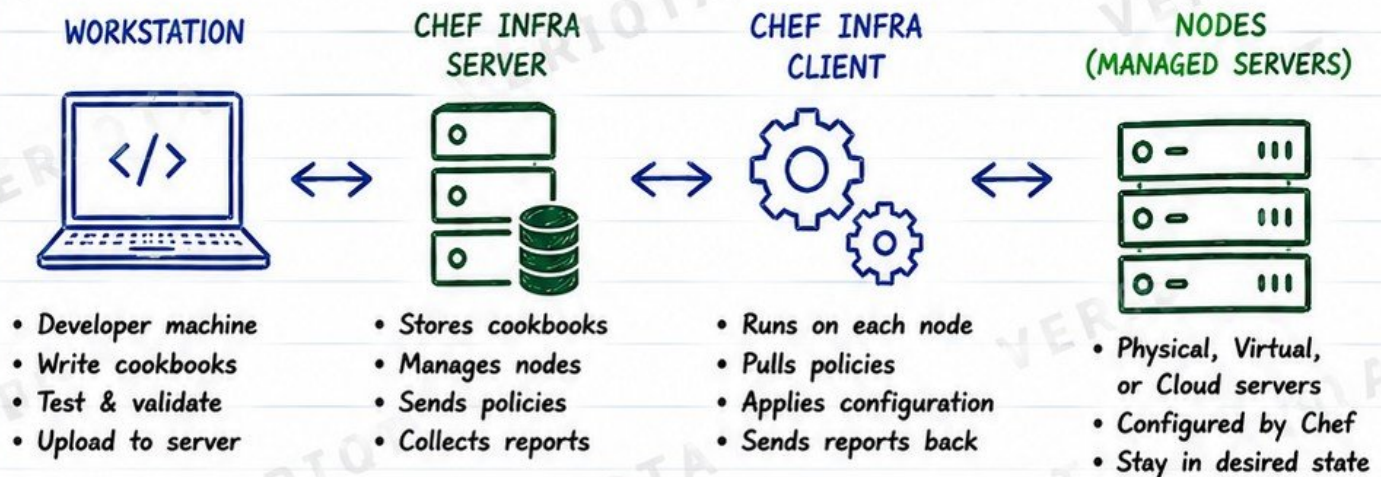


SENIOR ENGINEER TIP

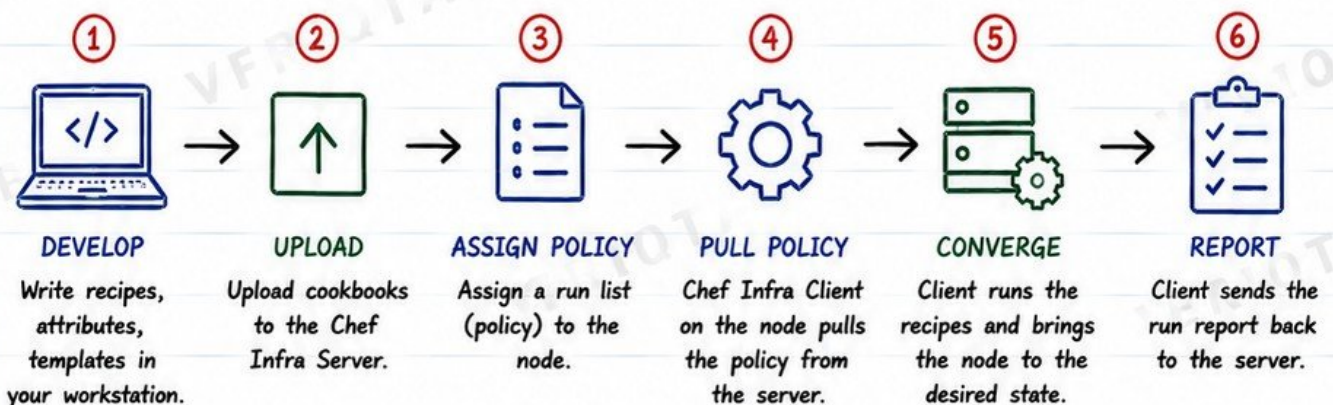
Always automate everything that can be automated. The goal is not just speed, it is consistency and reliability.

2. HOW CHEF WORKS

☆ CHEF INFRA ARCHITECTURE



☆ THE CONFIGURATION-MANAGEMENT FLOW



KEY TAKEAWAYS

- You define the desired state in cookbooks.
- Chef Server manages and distributes configuration.
- Chef Client applies configuration on nodes.
- The system stays consistent, repeatable, and automatic.



REAL WORLD LESSON

Chef does not push changes randomly. Nodes request their configuration, which makes the system secure, scalable, and reliable.

3. SETTING UP YOUR CHEF LAB

☆ BUILDING A SAFE PRACTICE ENVIRONMENT

CHEF WORKSTATION



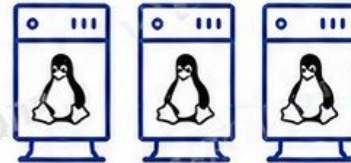
- You develop cookbooks
- Run Chef CLI
- Upload to server
- Test and validate

CHEF SERVER



- Stores cookbooks
- Manages nodes
- Sends policies
- Collects run reports

LINUX TEST NODES



- Managed by Chef Client
- Apply configuration
- Stay in desired state
- Send reports back



This lab is for learning. Isolated, safe and repeatable.

1) CHEF WORKSTATION



Your development machine. Write, test and upload cookbooks.
OS: macOS, Linux or Windows (WSL).

2) LINUX TEST NODES



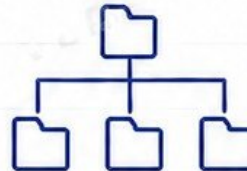
Use 2–3 lightweight Linux VMs.
OS: Ubuntu / Rocky / AlmaLinux (Server edition).

3) CHEF CLI



Command-line tool to manage Chef. Comes with Chef Workstation. You'll use it every day.

4) PROJECT STRUCTURE (LOCAL)



```
my-cookbooks/
├── cookbooks/
├── environments/
├── roles/
├── data_bags/
├── Berksfile
└── README.md
```

5) VERIFYING THE INSTALLATION

CHEF WORKSTATION

```
$ chef --version
$ knife --version
```

CHEF SERVER

```
$ knife status
$ knife node list
```

TEST NODE (AFTER BOOTSTRAP)

```
$ sudo chef-client -v
$ sudo chef-client
```



All commands should return version info or success messages. If not, check network, firewall and server URL.



LAB SETUP TIPS

- ✓ Use VirtualBox, VMware or UTM.
- ✓ Give each VM 2 vCPU / 2GB RAM minimum.
- ✓ Use private network / host-only network.
- ✓ Disable firewalls for lab (or open 443).
- ✓ Take snapshots before major changes.
- ✓ Keep your Chef Server and nodes isolated.



SAFETY FIRST

- ✓ Never practice directly on production.
- ✓ Keep your experiment isolated.
- ✓ Break things, learn, then rebuild.
- ✓ Documentation is your safety net.



REAL WORLD LESSON

A safe lab today prevents expensive mistakes in production tomorrow.

4. CHEF RESOURCES, YOUR FIRST AUTOMATION

★ WHAT A RESOURCE REPRESENTS



A RESOURCE IS CHEF'S WAY OF MANAGING A SINGLE PIECE OF YOUR SYSTEM.

CHEF ENSURES THE RESOURCE IS ALWAYS IN THE DESIRED STATE.

★ COMMON CORE RESOURCES

PACKAGE



ENSURES A PACKAGE IS INSTALLED (OR REMOVED).

FILE



MANAGES FILE CONTENT, OWNERSHIP, AND PERMISSIONS.

DIRECTORY



ENSURES A DIRECTORY EXISTS WITH CORRECT PERMS.

SERVICE



MANAGES SERVICE STATE (RUNNING, STOPPED, RESTARTED, ENABLED, DISABLED).

USER



MANAGES SYSTEM USERS.

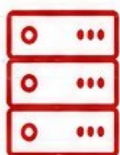
TEMPLATE



CREATES FILES FROM TEMPLATES WITH DYNAMIC CONTENT.

★ DESIRED-STATE THINKING

CURRENT STATE



DESIRED STATE



YOU DON'T TELL CHEF HOW TO GET THERE. YOU TELL CHEF WHAT THE END STATE SHOULD LOOK LIKE. CHEF FIGURES OUT THE REST.

★ FIRST WORKING RECIPE

1. CREATE A RECIPE: [recipes/hello.rb](#)

```
1 package 'nginx' do
2   action :install
3 end
4
5 service 'nginx' do
6   action [:enable, :start]
7 end
```

2. RUN: `sudo chef-client`

3. VERIFY: nginx is installed and running.

4. RE-RUN: Nothing changes if already correct.



KEY TAKEAWAY

- ✓ RESOURCES DECLARE THE DESIRED STATE.
- ✓ CHEF MAKES SURE THE ACTUAL STATE MATCHES IT.
- ✓ THIS IS HOW YOU GET CONSISTENT, REPEATABLE, AND RELIABLE SYSTEMS.

5. RECIPES AND DECLARATIVE CONFIGURATION

★ RECIPE STRUCTURE



recipes/
example.rb

- ① COMMENT / DESCRIPTION
- ② RESOURCE DECLARATIONS
- ③ PROPERTIES (OPTIONS)
- ④ ACTIONS
- ⑤ END OF RESOURCE

★ RESOURCE DECLARATION

```
package 'nginx' do
  action :install
end
```

- RESOURCE TYPE
- RESOURCE NAME
- ACTION
- END

★ PROPERTIES (OPTIONS)



- Define how the resource should be.
- Examples: version, owner, mode, path, content, enable, etc.
- Make the resource match your desired state.

★ ACTIONS



- What Chef should do with the resource.
- Common actions: :install, :start, :enable, :create, :delete, :restart.
- Default action is chosen if not specified.

★ COMPILE AND CONVERGE CONCEPTS

1) COMPILE PHASE



- Chef reads the recipe.
- Builds a "cookbook" in memory.
- No changes are made to the system.



2) CONVERGE PHASE



- Chef compares desired state vs actual state.
- Makes only the changes needed.
- System moves to desired state.



3) REPORT PHASE



- Chef sends the run results to the server.
- You can review logs and compliance.



KEY IDEA: CHEF NEVER REPEATS WORK UNNECESSARILY.
IT ENSURES YOUR SYSTEM MATCHES THE DESIRED STATE.

★ WHY CHEF RECIPES ARE NOT ORDINARY SHELL SCRIPTS

SHELL SCRIPTS



- Imperative (step-by-step commands)
- Runs every line every time
- No idea of current state
- Not idempotent by default
- Hard to scale and maintain

VS



CHEF RECIPES

- Declarative (define the desired state)
- Runs only what is necessary
- Knows the current state
- Idempotent by design
- Scalable, testable and repeatable



REAL WORLD LESSON

CHEF DOESN'T CARE HOW TO GET THERE.
IT CARES THAT THE END STATE IS CORRECT.



COMMON MISTAKE

THINKING YOU NEED SHELL COMMANDS FOR EVERYTHING.
USE RESOURCES FIRST. THEY ARE SAFER AND SMARTER.

6. COOKBOOKS FROM THE GROUND UP

☆ WHAT IS A COOKBOOK?



- A COOKBOOK IS A COLLECTION OF RECIPES, TEMPLATES, FILES AND ATTRIBUTES.
- IT DEFINES HOW YOUR SYSTEMS SHOULD BE CONFIGURED.
- COOKBOOKS MAKE YOUR AUTOMATION REUSABLE, ORGANIZED AND SCALABLE.

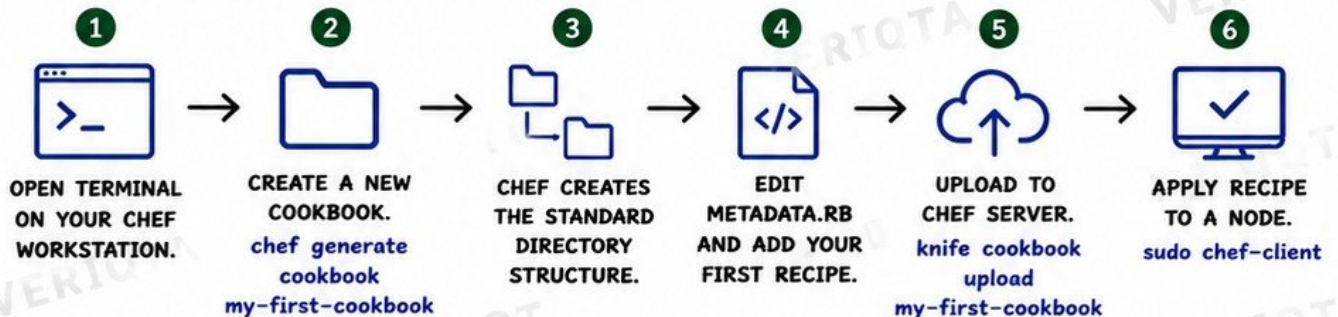
☆ COOKBOOK ANATOMY



☆ ANATOMY EXPLAINED

	METADATA.RB	Defines cookbook information and dependencies. Tells Chef what this cookbook needs.
	RECIPES/	Contains recipes. Recipes declare the resources that configure your system.
	TEMPLATES/	Stores template files (usually ERB). Used to generate dynamic configuration files.
	FILES/	Stores static files. Copied to the target node as-is.
	ATTRIBUTES/	Contains default attributes. Used to store configuration data for your cookbook.

☆ CREATING YOUR FIRST COOKBOOK



KEY TAKEAWAY

COOKBOOKS ARE THE FOUNDATION OF INFRASTRUCTURE AS CODE IN CHEF. ORGANIZE WELL, REUSE EVERYTHING.



BEST PRACTICE

- ✓ KEEP COOKBOOKS SMALL AND FOCUSED.
- ✓ USE ATTRIBUTES FOR CONFIGURATION.
- ✓ TEST BEFORE YOU DEPLOY.

7. NODES AND THE CHEF CONVERGENCE MODEL

☆ WHAT CHEF CALLS A NODE



- A node is any system managed by Chef.
- Physical server, VM, container, or cloud instance.
- Each node has its own identity and configuration.
- Chef stores node data on the Chef Server.

NODE REGISTRATION - HOW A NODE JOINS CHEF



WHAT HAPPENS

- ✓ Node gets unique ID
- ✓ Node data is stored on Chef Server
- ✓ Node is now manageable

CHEF INFRA CLIENT RUNS



- Chef Infra Client runs on the node.
- By default every 30 minutes (configurable).
- Pulls configuration from Chef Server.
- Applies configuration to reach desired state.

DEFAULT RUNS

- Interval: every 30 minutes
- Can be changed to suit your environment.
- More frequent = faster correction but more server load.

DESIRED VS CURRENT STATE

DESIRED STATE
(In Chef Server)



What should be true

VS

CURRENT STATE
(On the Node)



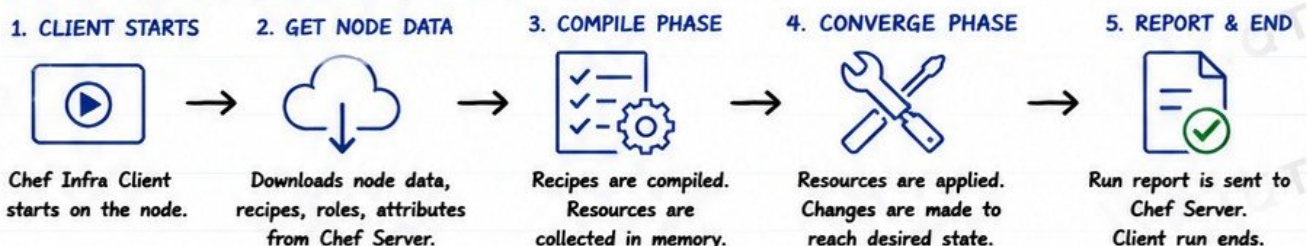
What is actually true

CONVERGENCE



- Chef compares desired vs current state.
- Makes only the changes needed.
- After run completes, system is in desired state.

WHAT HAPPENS DURING A CHEF RUN



PRODUCTION NOTE

Chef does not just run commands. It enforces desired state. If something drifts, Chef fixes it automatically on the next run.



SENIOR ENGINEER TIP

Keep your runs idempotent, fast, and safe. If a run is flaky or slow, engineers will lose trust in automation.

8. ATTRIBUTES AND CONFIGURATION DATA

☆ WHY HARD-CODED VALUES BECOME A PROBLEM



- Values get duplicated in many recipes.
- Harder to update and maintain.
- Different environments need different values.
- Leads to errors and inconsistent systems.

BAD EXAMPLE

package 'nginx' do
version '1.20.1'
end

Now you must change it everywhere by hand.

TYPES OF ATTRIBUTES



NODE ATTRIBUTES

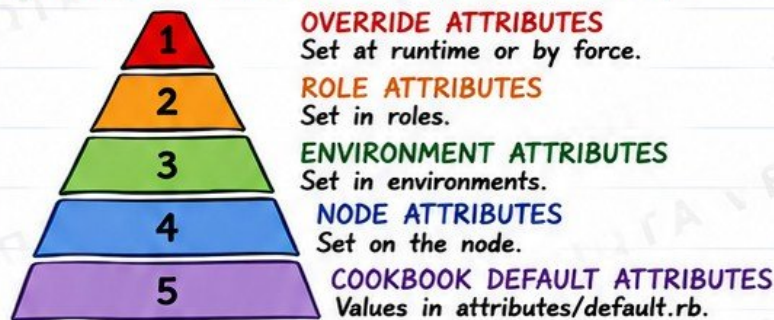
- Specific to a node.
- Defined on the node object.
- Example: IP, memory, OS, data center.



COOKBOOK ATTRIBUTES

- Defined inside a cookbook.
- Provide default values.
- Can be overridden.
- Make recipes reusable.

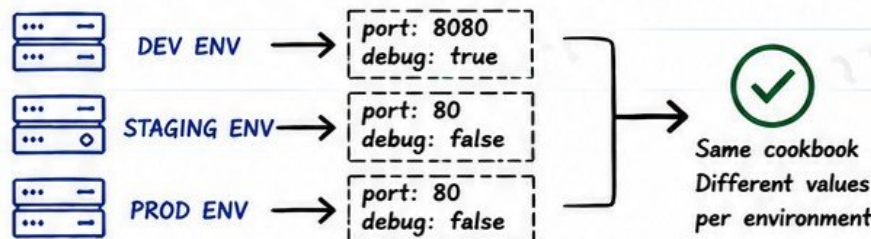
ATTRIBUTE PRECEDENCE (HIGH TO LOW)



KEY IDEA

Higher level always wins over lower level. This allows flexibility without changing cookbook code.

ENVIRONMENT-SPECIFIC CONFIGURATION



BEST PRACTICE

Keep code the same. Change only data. This is the power of attributes.

REUSABLE RECIPES WITH ATTRIBUTES



RECIPE

```
node['web']['port']
node['nginx']['user']
node['nginx']['worker_processes']
```

- ✓ Same recipe used everywhere.
- ✓ Values come from attributes.
- ✓ No hard-coded values.
- ✓ Easy to maintain and scale.



REUSABLE & MAINTAINABLE



PRODUCTION NOTE

Well-structured attributes make your cookbooks flexible, predictable, and easy to scale across environments.



SENIOR ENGINEER TIP

Never hard-code values in recipes. Use attributes, roles, and environments. Your future self will thank you.

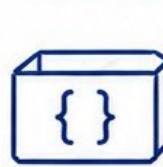
9. TEMPLATES AND DYNAMIC CONFIGURATION FILES

☆ ERB TEMPLATES



- ERB (Embedded Ruby) lets you insert dynamic values into configuration files.
- Chef renders the template and writes the final file on the node.

☆ VARIABLES



node['ipaddress']
node['port']
node['fqdn']
@app_name
@listen_port

Values come from attributes, node data, or recipe variables.

☆ GENERATING CONFIGURATION FILES



TEMPLATE



CHEF RENDERS



FINAL FILE



ON NODE

EXAMPLE

nginx.conf.erb



/etc/nginx/nginx.conf

☆ FILE OWNERSHIP AND PERMISSIONS



Always control who can read and change your config files.

```
template '/etc/nginx/nginx.conf' do
  source 'nginx.conf.erb'
  owner 'root'
  group 'root'
  mode '0644'
end
```

☆ NOTIFICATIONS



- Restart or reload services only when a file actually changes.
- Prevents unnecessary restarts.

```
template '/etc/nginx/nginx.conf' do
  source 'nginx.conf.erb'
  notifies :reload, 'service[nginx]'
end
service 'nginx' do
  action :nothing
end
```

☆ MANAGING AN NGINX CONFIGURATION WITH CHEF

ATTRIBUTES (attributes/default.rb)

```
default['nginx']['listen_port'] = 80
default['nginx']['server_name'] = 'example.com'
default['nginx']['root'] = '/var/www/html'
default['nginx']['user'] = 'www-data'
```

WHAT HAPPENS

1. Chef reads attributes.
2. Renders nginx.conf.erb.
3. Writes /etc/nginx/nginx.conf.
4. If file changed → reload nginx.

nginx.conf.erb (template)

```
user <%= node['nginx']['user'] %>;
worker_processes auto;
events { worker_connections 1024; }
http {
  server {
    listen <%= node['nginx']['listen_port'] %>;
    server_name <%= node['nginx']['server_name'] %>;
    root <%= node['nginx']['root'] %>;
  }
}
```



PRODUCTION NOTE

Templates keep configs consistent across servers. Change data, not code. This is how you scale safely.

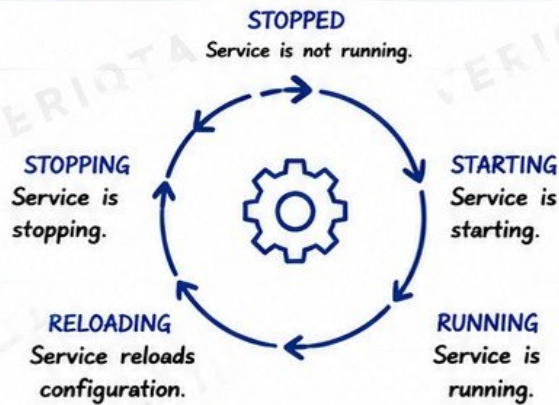


SENIOR ENGINEER TIP

Use attributes for all values. Avoid hard-coding anything. Clean templates + good attributes = maintainable infrastructure.

10. SERVICES, NOTIFICATIONS, AND DEPENDENCIES

SERVICE LIFECYCLE



IMMEDIATE VS DELAYED NOTIFICATIONS



IMMEDIATE

- Action happens immediately.
- Good for quick changes.
- Can cause service flapping.



DELAYED

- Action happens once at the end of the run.
- Good for multiple changes.
- Safer for production.

NOTIFIES (TRIGGER FROM THE RESOURCE)



Resource

--notifies-->



Service

- When resource changes, it notifies the service.
- Example: Config file changes → restart service.

```
template '/etc/nginx/nginx.conf' do
  source 'nginx.conf.erb'
  notifies :restart, 'service[nginx]'
end
```

SUBSCRIBES (LISTEN FROM THE SERVICE)



Resource

<--subscribes--



Service

- Service listens for changes from resources.
- Service takes action when notified.

```
service 'nginx' do
  action :nothing
  subscribes :restart, 'template[/etc/nginx/nginx.conf]'
end
```

RESTARTING ONLY WHEN CONFIGURATION CHANGES



Config File

Change



Service

- Chef checks file content (not just timestamps).
- If content is different → notification triggers.
- Service restarts only when needed.

★ BEST PRACTICE

Always use notifications to keep services stable. Restart only on real changes.

BUILDING SAFE DEPENDENCY CHAINS


1. TEMPLATE
CREATE CONFIG

2. NOTIFY
SERVICE

3. SERVICE
RESTARTS

4. SERVICE
RUNNING

EXAMPLE CHAIN

- ✓ Config template changes
- ✓ Notifies nginx service
- ✓ Nginx restarts once at end
- ✓ Application stays available



PRODUCTION NOTE

- Poor notifications = unstable services.
- Too many restarts = downtime.
- Build chains that are predictable, idempotent, and safe.



SENIOR ENGINEER TIP

- Use delayed notifications for services.
- Use immediate notifications for one-off tasks.
- Always test before pushing to production.

11. OHAI AND AUTOMATIC SYSTEM DISCOVERY

☆ WHAT OHAI DOES



Ohai is Chef's built-in system profiler. It automatically discovers details about the system and makes them available as node attributes.

WHY IT MATTERS

- ✓ No manual fact collection.
- ✓ Adapts recipes to each system.
- ✓ Makes cookbooks dynamic and portable.
- ✓ Reduces hard-coding and duplication.

INFORMATION OHAI DISCOVERS

OS & PLATFORM



- Operating system
- Platform family
- Platform version
- Kernel version
- Architecture

IP ADDRESSES



- IPv4 addresses
- IPv6 addresses
- Default gateway
- MAC address

CPU & MEMORY



- CPU model
- CPU cores
- CPU speed
- Total memory
- Memory usage

NETWORK INTERFACES



- Interface names
- IP addresses
- MAC addresses
- Netmask
- MTU

OTHER FACTS



- Disk information
- Load averages
- Uptime
- Virtualization
- Many more...

SEEING OHAI IN ACTION

```
$ ohai
...
$ ohai platform
platform          ubuntu
platform_version  22.04
platform_family   debian
platform_release  jammy
...
```

REAL WORLD LESSON

Ohai is the reason Chef can write one cookbook and run it on many different systems.

It removes guesswork and makes automation intelligent.

USING DISCOVERED INFORMATION IN RECIPES

```
node['kernel']['machine'] => x86_64
node['platform']          => ubuntu
node['platform_version']  => 22.04
node['ipaddress']         => 192.168.1.10
node['memory']['total']   => 16324544
node['network']['interfaces']['eth0']['ipaddress']
=> 192.168.1.10
```



```
package 'nginx'
  action :install
end

template '/etc/nginx/conf.d/info.conf' do
  source 'info.conf.erb'
  variables({
    :ip => node['ipaddress'],
    :platform => node['platform']
  })
end
```



PRODUCTION NOTE

Ohai data is collected at the start of every Chef run. If something changes outside of Chef, the next run will use the new facts.



SENIOR ENGINEER TIP

Use Ohai facts in attributes, recipes, templates, and conditionals to build smart, dynamic, environment-aware automation.

12. CHEF SERVER AND CENTRALIZED MANAGEMENT

★ WHY CENTRALIZED MANAGEMENT MATTERS

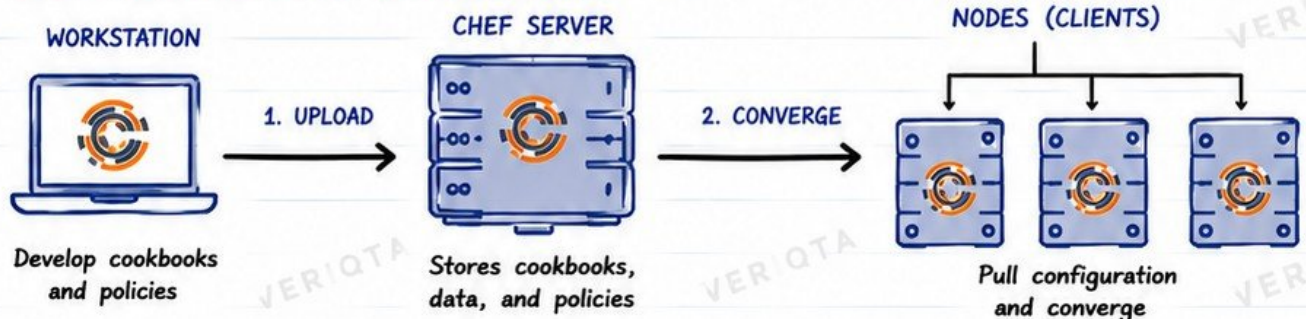


- Single source of truth for cookbooks and data.
- Consistent configuration across all nodes.
- Secure authentication and access control.
- Scales from a few servers to thousands.
- Auditability, visibility, and compliance.

REAL WORLD LESSON

Without a Chef Server, each machine runs in isolation. With Chef Server, your entire infrastructure works as one managed system.

CHEF ARCHITECTURE (CENTRALIZED)



① UPLOADING COOKBOOKS



- Cookbooks are versioned.
- Upload using chef CLI or Workstation.
- Stored on Chef Server for all nodes to use.

```
$ knife cookbook upload my_cookbook
Uploading my_cookbook [1.0.0]
Uploaded 1 cookbook.
```

② REGISTERING NODES



- Nodes are registered with the Chef Server.
- Each node gets a unique identity.
- Node name = FQDN.

```
$ sudo chef-client -s chef.example.com
Node registration['web01']
Chef Infra Client, version 18.x
```

③ AUTHENTICATION



- Secure access with client key pairs.
- Only trusted nodes can communicate.
- Protects cookbooks and data.

```
$ knife node list
Enter Client Key passphrase:
web01.example.com
```

④ CHEF SERVER COMMUNICATION



- Nodes connect over HTTPS.
- Pull cookbooks, policies, and data.
- Send run results and status back.
- Communication is encrypted.

```
Node ----> Chef Server (Pull)
Node <---- Chef Server (Report)
```

⑤ MANAGING MULTIPLE MACHINES



- Manage hundreds or thousands of nodes.
- Assign roles and environments.
- Apply changes at scale with confidence.

⑥ CHEF CLI ESSENTIALS



- knife cookbook list
- knife node list
- knife environment list
- knife status
- knife data bag list

```
$ knife status
chef.example.com 3 nodes
web01            successful
web02            successful
db01             successful
```



PRODUCTION NOTE

Your Chef Server is critical. Back it up, monitor it, and secure it like any other production system.



SENIOR ENGINEER TIP

Keep cookbooks small, reusable, and well tested. Centralization gives power—use it with discipline!

13. ROLES, ENVIRONMENTS, AND INFRASTRUCTURE ORGANIZATION

☆ 1. WEB, APPLICATION, AND DATABASE ROLES

WEB ROLE



- Handles HTTP/HTTPS
- Nginx, Apache, HAProxy
- Static content
- Load balancing

APPLICATION ROLE



- Runs business logic
- App servers (Tomcat, Java, Node.js, etc.)
- Processes requests
- Connects to databases

DATABASE ROLE



- Stores data
- MySQL, PostgreSQL, MongoDB, etc.
- Backups and replication
- High availability

2. DEVELOPMENT, STAGING, PRODUCTION ENVIRONMENTS

DEVELOPMENT



- For developers
- Fast changes
- Less strict security
- Can break things
- Used for learning

STAGING



- Pre-production testing
- Mirrors production
- Validate changes
- QA and UAT testing
- Should be as close to production as possible

PRODUCTION



- Live users
- Stable and secure
- High availability
- Monitored and backed up
- Changes are controlled and minimal

3. ORGANIZING FLEETS (NODES)



- Fleets are groups of nodes with the same role and purpose.
- Example:

Web Fleet
App Fleet
DB Fleet

BENEFITS

- ✓ Easier management
- ✓ Consistent configuration
- ✓ Simple targeting
- ✓ Scales cleanly
- ✓ Clear separation of concerns

4. ENVIRONMENT DIFFERENCES (EXAMPLES)

SETTING	DEVELOPMENT	STAGING	PRODUCTION
LOG LEVEL	DEBUG	INFO	WARN / ERROR
RESOURCE SIZE	SMALL	MEDIUM	LARGE
SECURITY	BASIC	STRONGER	STRICT
BACKUPS	OPTIONAL	YES	MANDATORY
CHANGE FREQUENCY	FREQUENT	OCCASIONAL	RARE & CONTROLLED
MONITORING	BASIC	FULL	FULL + ALERTING

REAL WORLD LESSON

Treat production as if failure will happen.
Design for recovery, not hope.

5. WHY EXCESSIVE ROLE COMPLEXITY IS DANGEROUS



- Too many roles create tight coupling.
- Small changes affect many places.
- Harder to test and maintain.
- More cookbooks, more dependencies.
- Increases risk of configuration drift.
- Slows down automation and delivery.

KEEP IT SIMPLE

- ✓ Create roles that match responsibilities.
- ✓ Avoid one-role-per-server syndrome.
- ✓ Reuse cookbooks and attributes.
- ✓ Design for clarity, not cleverness.



14. DATA BAGS, SECRETS, AND CONFIGURATION DATA

1. DATA BAGS



- Stores structured data.
- Separate from nodes and cookbooks.
- Shared across environments.
- Great for lookup information.

EXAMPLE DATA BAG (JSON)

```
{
  "id": "db",
  "engine": "postgresql",
  "host": "db01.example.com",
  "port": 5432,
  "username": "appuser"
}
```

REAL WORLD LESSON



Data bags let you change values without changing code. One change in data, every node gets it. That is power and safety.

2. SHARED CONFIGURATION



- Store common values once.
- Use in multiple recipes and environments.
- Avoid duplication.
- Easier updates and scaling.

USE CASES

- ✓ API endpoints
- ✓ License keys
- ✓ Feature flags
- ✓ Email settings
- ✓ Database connection info

3. SENSITIVE INFORMATION



- Passwords
- API keys
- Tokens
- Private keys
- Certificates

4. WHY SECRETS SHOULD NOT LIVE IN RECIPES



- Code goes to source control.
- Anyone with access can read it.
- Hard to rotate secrets.
- Increases security risk.
- Violates compliance policies.

BAD EXAMPLE

```
user 'deploy' do
  password 'P@ssw0rd123'
end
```



5. SECRET-MANAGEMENT CONSIDERATIONS

ENCRYPT SECRETS



Use Chef Encrypted Data Bags or external secret stores.

LIMIT ACCESS



Only authorized users and nodes should access secrets.

ROTATE REGULARLY



Change secrets periodically. Reduce exposure.

AUDIT USAGE



Know where secrets are used and by which nodes.

USE PROPER TOOLS



Vault, AWS Secrets Manager, Azure Key Vault, CyberArk, etc.

6. SEPARATING CODE FROM DATA



- Code = how things are built.
- Data = the values that change.
- Keep them separate.
- Easier to maintain.
- Safer for production.

CODE (COOKBOOK)



Recipes, resources, templates, logic, structure.

+

DATA (DATA BAGS)



Values, credentials, settings, endpoints, shared info.

=

CONFIDENT & FLEXIBLE AUTOMATION



PRODUCTION NOTE

Treat secrets like keys to your home. If leaked, assume compromise. Protect them like your infrastructure depends on it—because it does.



SENIOR ENGINEER TIP

Great automation is not just about code. It is about safe data, clear boundaries, and secure operations.

15. MANAGING SERVERS WITH CHEF

CHEF HELPS YOU KEEP SERVERS CONSISTENT, SECURE, AND RELIABLE. EVERY TIME.

1. USERS AND GROUPS



- Create users
- Manage groups
- Set home directories
- Enforce SSH access

2. PACKAGES



- Install packages
- Remove packages
- Keep versions
- Cross-platform support

3. FILES AND DIRECTORIES



- Create files and folders
- Set permissions
- Deploy configuration files
- Ensure correct ownership

4. SERVICES



- Start, stop, enable services
- Restart when needed
- Ensure services are running

5. SSH CONFIGURATION



- Manage SSHD config
- Allow only key auth
- Disable root login
- Control SSH access safely

6. SCHEDULED TASKS



- Create cron jobs
- Run commands on schedule
- Maintain backups, cleanups, reports

7. BUILDING A BASELINE SERVER COOKBOOK



- Define your standard
- Reusable and versioned
- Apply to all new servers
- Foundation for all environments

EXAMPLE: MANAGING A SERVER WITH CHEF

CREATE USER

```
user 'deploy' do
  comment 'Deployment User'
  home '/home/deploy'
  shell '/bin/bash'
  supports:manage_home => true
end
```

INSTALL PACKAGE

```
package 'nginx' do
  action :install
end
```

MANAGE FILE

```
file '/etc/motd' do
  content 'Managed by Chef
Always Automate!'
  owner 'root'
  group 'root'
  mode '0644'
end
```

MANAGE SERVICE

```
service 'nginx' do
  action [:enable, :start]
end
```

CONFIGURE SSH

```
template '/etc/ssh/sshd_config' do
  source 'sshd_config.erb'
  owner 'root'
  group 'root'
  mode '0600'
  notifies:restart, 'service[sshd]'
end
```

SCHEDULED TASK (CRON)

```
cron 'cleanup_tmp' do
  minute '0'
  hour '2'
  command 'rm -rf /tmp/*'
  user 'root'
end
```

REAL WORLD LESSON



A baseline cookbook ensures every server starts with the same secure settings, tools, users, and services. It reduces drift and saves hours of manual work.

BASELINE SERVER CHECKLIST

- ✓ Users and groups are correct
- ✓ Required packages are installed
- ✓ Files and directories are in place
- ✓ Services are running and enabled
- ✓ SSH is locked down
- ✓ Scheduled tasks are set
- ✓ Server is consistent and secure



PRODUCTION NOTE

Automate the boring. Standardize the important. Chef keeps your servers predictable and safe.



SENIOR ENGINEER TIP

Build small, reusable cookbooks. Test them. Version them. Then scale your infrastructure with confidence.

16. CHEF AND NETWORKING CONFIGURATION



NETWORKING IS CRITICAL. ONE WRONG CHANGE
CAN DISCONNECT YOU FROM YOUR SERVERS.
AUTOMATE CAREFULLY. TEST SAFELY. ROLLBACK FAST.

1. HOSTNAMES



- Set consistent, meaningful hostnames.
- Used for identification, logging, monitoring, and DNS.
- Keep FQDN and short name in sync.

RECIPE EXAMPLE

```
hostname node['network']['fqdn']
notifies :run, 'execute[update-hosts]'; :immediately
execute 'update-hosts' do
  command 'echo 127.0.0.1 localhost >> /etc/hosts'
  action :nothing
end
```

2. DNS-RELATED CONFIGURATION



- Configure resolvers (nameservers) correctly.
- Search domains control short-name resolution.
- Wrong DNS breaks everything.

/etc/resolv.conf TEMPLATE (ERB)

```
nameserver <%= node['dns']['primary'] %>
nameserver <%= node['dns']['secondary'] %>
search <%= node['dns']['search'] %>
```

3. NETWORK CONFIGURATION FILES



- Manage IP, gateway, DNS, and interfaces.
- Cloud environments may use DHCP.
- Traditional servers often use static configuration.

EXAMPLE: /etc/netplan/01-netcfg.yaml (UBUNTU)

```
network:
  version: 2
  ethernet:
    ens33:
      addresses: [192.168.1.20/24]
      gateway4: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

4. FIREWALL RULES



- Default deny inbound.
- Allow only what is required.
- Keep rules simple and documented.

CHEF FIREWALL (UFW) EXAMPLE

```
ufw_rule 'allow_ssh' do
  rule 'allow'
  port 22
  proto 'tcp'
end
ufw_rule 'allow_http' do
  rule 'allow'
  port 80
  proto 'tcp'
end
```

PRO TIP

Always keep SSH access open from your IP or a bastion host. Never lock yourself out!

5. PORTS AND SERVICES



- Every service uses a port.
- Know what is listening.
- Don't expose unnecessary ports.

COMMON PORTS	SERVICE
22	SSH
53	DNS
80	HTTP
443	HTTPS
3306	MySQL
5432	PostgreSQL

6. SAFE NETWORK CHANGES



- Test changes before applying everywhere.
- Use notifications to apply step by step.
- Validate before reloading services.
- Keep a rollback plan.



PLAN
CHANGE



TEST
LOCALLY



VALIDATE
CONFIG



APPLY
CHANGE



VERIFY
ACCESS

7. AVOIDING REMOTE LOCKOUTS



- Always have a backup access method (console, cloud session, or out-of-band).
- Don't reload network services until you confirm the new config is valid.
- Use automation with extreme caution when changing SSH, firewall, or routing.
- Keep old config backups. Be ready to revert.



GOLDEN RULE

AUTOMATE EVERYTHING
EXCEPT LOSING ACCESS.

17. TESTING CHEF INFRASTRUCTURE



TESTING INFRASTRUCTURE CODE IS NOT OPTIONAL.
A SMALL TEST TODAY PREVENTS A BIG OUTAGE TOMORROW.

1. WHY INFRASTRUCTURE CODE NEEDS TESTING



- Infrastructure changes can break systems.
- Mistakes are expensive in production.
- Tests give confidence to deploy safely.
- Catch errors early and save time.
- Make refactoring safe and repeatable.

2. CHEFSPEC CONCEPT



- Unit testing for Chef recipes.
- Tests the "compile" phase only.
- Ensures resources are declared correctly.
- Fast, lightweight, runs locally.
- Written in Ruby with RSpec.

CHEFSPEC EXAMPLE

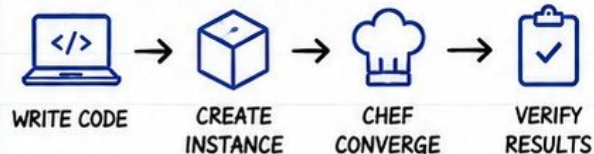
```
describe 'myapp::default' do
  it 'installs nginx package' do
    is_expected.to install_package('nginx')
  end
end
```

3. TEST KITCHEN



- Integration testing tool for Chef.
- Spins up real or virtual machines.
- Runs the full Chef converge.
- Verifies system behavior.
- Supports multiple platforms.

TEST KITCHEN FLOW



4. INSPEC INTRODUCTION



- Compliance and validation framework.
- Tests the final state of the system.
- Can run locally or in CI/CD.
- Uses simple, readable controls.
- Works on any infrastructure.

INSPEC EXAMPLE

```
describe port(80) do
  it { should be_listening }
end
describe file('/etc/nginx/nginx.conf') do
  it { should exist }
end
```

5. LOCAL CONVERGENCE TESTING



- Use Vagrant, Docker, or VirtualBox.
- Run Chef locally before pushing changes.
- See real results on a real system.
- Safe place to experiment and debug.
- Repeatable and fast feedback loop.

EXAMPLE WORKFLOW

```
$ kitchen create
$ kitchen converge
$ kitchen verify
$ kitchen destroy
```

6. CATCHING FAILURES BEFORE PRODUCTION



- Prevent broken deployments.
- Avoid manual rollbacks.
- Protect users and services.
- Build a culture of safe changes.
- Test everything. Trust nothing.

PRO TIP

AUTOMATE YOUR TESTS.
RUN THEM ON EVERY CHANGE.
NEVER DEPLOY UNTESTED CODE.



TESTING IS HOW YOU PROVE YOUR INFRASTRUCTURE CODE IS
CORRECT, SAFE, AND PRODUCTION READY.
TEST EARLY. TEST OFTEN. DEPLOY WITH CONFIDENCE.

18. CHEF IN CLOUD INFRASTRUCTURE



THE CLOUD IS DYNAMIC. CHEF BRINGS CONSISTENCY.
PROVISION FAST. CONFIGURE RIGHT. SCALE CONFIDENTLY.

1. CHEF WITH AWS, AZURE, AND CLOUD VMS



- EC2 Instances
- Auto Scaling Groups
- IAM Roles
- User Data / Bootstrap



- Virtual Machines
- Scale Sets
- Managed Identities
- Custom Script Ext.



- Compute Engine
- Instance Groups
- Service Accounts
- Startup Scripts

2. PROVISIONING VS CONFIGURATION

PROVISIONING
(Create It)



- Build instances
- Networks
- Storage
- Firewalls



CONFIGURATION
(Make It Right)



- Install software
- Configure services
- Manage users
- Enforce policies

PROVISIONING CREATES. CHEF CONFIGURES AND MAINTAINS.

3. TERRAFORM + CHEF



TERRAFORM
Provision
Infrastructure



CLOUD
Instances
Created



CHEF
Configure &
Manage



DESIRED STATE
Consistent &
Compliant

TERRAFORM BUILDS. CHEF CONFIGURES.
TOGETHER THEY DELIVER RELIABLE INFRASTRUCTURE.

4. BOOTSTRAPPING CLOUD INSTANCES



1. Instance is provisioned.
2. Cloud-init / User Data runs.
3. Installs Chef Infra Client.
4. Sets server URL and token.
5. Node registers with Chef Server.
6. Chef run applies the desired state.

EXAMPLE: USER DATA (AWS)

```
#!/bin/bash
curl -L https://omnitruck.chef.io/install.sh | sudo bash
sudo chef-client \
  -o "[my_cookbook]::default" \
  -s https://chef.example.com \
  -t YOUR_NODE_TOKEN
```

5. AUTOSCALING CONSIDERATIONS



- New instances must bootstrap automatically.
- Use launch templates with user data.
- Ensure Chef Server is highly available.
- Use roles and environments for scale.
- Keep runs lightweight and idempotent.

SCALE OUT = MORE NODES. NOT MORE MANUAL WORK.

6. EPHEMERAL INFRASTRUCTURE



- Instances are short-lived.
- No pets. Only cattle.
- Everything must be automated.
- Store no state on instances.
- Chef ensures fast, consistent rebuilds.

BUILD. CONFIGURE. RUN. DESTROY. REPEAT.

7. CLOUD WORKFLOW WITH CHEF



TERRAFORM
Provision
Infrastructure



CLOUD PROVIDER
Create VMs
Networks, Storage



BOOTSTRAP
User Data Installs
Chef Client



CHEF SERVER
Node Registers
Pulls Recipes



CONFIGURED NODE
Converges to
Desired State



SCALE / REPLACE
New Nodes
Bootstrap & Converge



AUTOMATE PROVISIONING. STANDARDIZE CONFIGURATION. SCALE WITH CONFIDENCE.
CHEF MAKES CLOUD INFRASTRUCTURE PREDICTABLE AND RELIABLE.



SENIOR ENGINEER TIP

Keep your Chef runs fast, idempotent, and stateless. Use attributes and roles wisely. Test your bootstrap scripts as carefully as your recipes.



PRODUCTION NOTE

A broken bootstrap or Chef Server outage can stop your entire fleet from configuring. Design for failure. Monitor everything.

19. PRODUCTION CHEF OPERATIONS AND TROUBLESHOOTING



PROBLEMS WILL HAPPEN. YOUR JOB IS TO FIND THE CAUSE, FIX IT SAFELY, AND PREVENT IT FROM HAPPENING AGAIN.

1. FAILED CHEF RUNS



- Resources failed
- Run was interrupted
- Node did not converge

2. COOKBOOK DEPENDENCY PROBLEMS



- Missing dependencies
- Version conflicts
- Berkshelf issues

3. AUTHENTICATION FAILURES



- Invalid client key
- Expired certs
- Server connection or SSL issues

4. TEMPLATE ERRORS



- ERB syntax errors
- Missing variables
- Invalid output

5. ATTRIBUTE SURPRISES



- Wrong precedence
- Unexpected values
- Hidden overrides
- Type mismatches

6. DEBUGGING CONVERGENCE



- Find the real root cause
- Understand resource flow
- Isolate the issue

7. LOGS ARE YOUR BEST FRIEND



- Chef client logs show everything.
- Always check logs first.
- Look for the real error, not the noise.

IMPORTANT LOG LOCATIONS (LINUX)

```
/var/log/chef/client.log
/var/log/chef/bootstrap.log
~/.chef/knife.log
/var/chef/cache/chef-stacktrace.out
```

8. USE CHEF DEBUG COMMANDS



- See more details about failures.
- Test changes safely.
- Understand what Chef is doing.

USEFUL COMMANDS

```
sudo chef-client -l info      # Normal detailed run
sudo chef-client -l debug     # Very detailed output
sudo chef-client -W           # Why-run (dry run)
knife node show <nodename>   # View node data
knife client list              # List all nodes
```

9. SAFE COOKBOOK CHANGES



- Test before production.
- Use version control (Git).
- Deploy in stages.
- Validate on a test node.
- Monitor every change.



TEST
LOCALLY



COMMIT
CHANGES



DEPLOY TO
TEST NODE



VALIDATE &
MONITOR



DEPLOY TO
PRODUCTION

10. ROLLBACK THINKING



- Always have a rollback plan.
- Keep previous working versions.
- Roll back fast when needed.
- Automation + backups save time.

ROLLBACK OPTIONS

- ✓ Revert Git commit
- ✓ Deploy previous cookbook version
- ✓ Use environment overrides
- ✓ Restore from backup (configs/data)
- ✓ Re-run last known good Chef run



TROUBLESHOOT LIKE A PROFESSIONAL

FIND THE ROOT CAUSE. FIX IT SAFELY. IMPROVE THE SYSTEM.
EACH PROBLEM IS A CHANCE TO MAKE YOUR INFRASTRUCTURE STRONGER.



SENIOR ENGINEER TIP

- Don't just fix it. Understand why it broke.
- Improve your cookbooks and processes.
- Good troubleshooting prevents repeat incidents.



PRODUCTION NOTE

- Never ignore a failed Chef run.
- Never bypass errors.
- Clean, consistent runs = reliable systems.

20. FINAL PROJECT: AUTOMATE A MULTI-SERVER ENVIRONMENT



YOUR MISSION: BUILD, AUTOMATE, AND OPERATE A COMPLETE MULTI-SERVER ENVIRONMENT USING CHEF.
AUTOMATE EVERYTHING. VALIDATE EVERYTHING. SHIP WITH CONFIDENCE.

1. PROVISION LINUX SERVERS



- Create VMs in your cloud (AWS / Azure / GCP).
- Web, App, DB, and Utility servers.
- Use consistent naming and tagging.

2. BOOTSTRAP CHEF NODES



- Install Chef Infra Client.
- Register nodes to Chef Server.
- Verify node runs and converges.
- Ensure secure communication.

3. BUILD REUSABLE COOKBOOKS



- Separate cookbooks by role.
- Use attributes, templates, and data bags.
- Make cookbooks reusable, modular, and scalable.

4. CONFIGURE USERS AND SECURITY BASELINES



- Create users and groups.
- Setup SSH hardening.
- Configure sudo rules.
- Apply system security baselines.

5. INSTALL AND CONFIGURE NGINX

NGINX

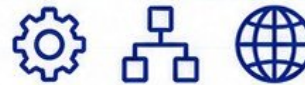
- Install NGINX package.
- Deploy custom configuration.
- Enable security headers.
- Manage service and logs.

6. MANAGE APPLICATION CONFIGURATION



- Use templates for app configs.
- Store environment-specific values in attributes/data bags.
- Keep secrets out of recipes.

7. CONFIGURE SERVICES AND NETWORKING



- Manage services and dependencies.
- Configure hostnames and /etc/hosts.
- Setup firewalls (UFW / iptables).
- Open only required ports.

8. TEST BEFORE DEPLOYMENT



- Use Test Kitchen + InSpec.
- Run local convergence tests.
- Validate configs and compliance.
- Fix issues early.

9. APPLY CONFIGURATION THROUGH CHEF SERVER



- Upload cookbooks to Chef Server.
- Assign roles and environments.
- Trigger Chef runs on all nodes.
- Monitor converge status.

10. INTRODUCE TERRAFORM + CHEF WORKFLOW



- Terraform provisions infrastructure.
- Chef configures and manages nodes.
- Keep infrastructure and configuration separated but integrated.
- Immutable infrastructure + consistent configuration.

11. TROUBLESHOOT CONFIGURATION DRIFT



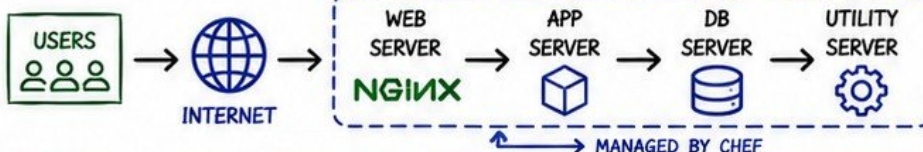
- Detect drift with Ohai and reports.
- Understand why drift happens.
- Fix at the source (cookbooks, attributes, or data).
- Re-run and ensure convergence.

12. VALIDATE THE FINAL INFRASTRUCTURE



- All servers converged.
- Services running correctly.
- Security baseline validated.
- Application accessible.
- Everything documented.
- You built it. You automated it. You own it.

SAMPLE MULTI-SERVER ARCHITECTURE



TECH STACK

- LINUX
- CHEF INFRA
- NGINX
- TERRAFORM
- AWS / AZURE / GCP
- TEST KITCHEN
- INSPEC
- GIT



THIS FINAL PROJECT BRINGS EVERYTHING TOGETHER.
INFRASTRUCTURE AS CODE + CONFIGURATION AS CODE = RELIABLE, REPEATABLE, SCALABLE.